2. Solution Overview

Team Name: Bharath Kumar Pallem | **Team Leader Name:** Bharath Kumar Pallem

Problem Statement: Build an offline AI Candidate Discovery & Ranking System to ingest, clean, and score 100,000 resume profiles and output the Top 100 candidate matches for the Senior AI Engineer role within 5 minutes on CPU.

We propose a **highly optimized, multi-stage filtering and heuristic scoring pipeline** built completely in offline Python. Rather than calling expensive external LLM APIs (which violate latency, offline, and cost limits at scale), our system inspects candidate profiles line-by-line, runs them through logic validation gates, and uses a calibrated log-scaled scoring algorithm to rank fits.

Traditional Keyword Matching Systems	Our AI-Native Ranking Approach
• Match exact keyword tokens (e.g. searching 'RAG' misses 'Dense Retrieval').	• **Weighted Skill Mapping:** Scores skills by logical synonyms (RAG/Retrieval-Augmented Generation mapped together) and scales dynamically.
• Susceptible to keyword stuffing (ranking fake or unqualified 'experts' highly).	• **Sanitizer Gateways:** Identifies and disqualifies profiles with impossible date timelines or empty expert durations.
• Ignores availability signals (ranking passive profiles who never log in).	• **Behavioral Multipliers:** Incorporates platform activity, response rates, and open-to-work flags as dynamic ranking multipliers.
• Blind to structural context (ranking candidates with services-only background).	• **Product Engineering Bias:** Disqualifies consulting-only profiles to align with the JD's founding team culture.

3. JD Understanding & Candidate Evaluation

Key Requirements Extracted from Job Description:

- **Core Expertise:** embeddings-based retrieval systems (dense/hybrid search) and vector databases (Pinecone, Weaviate, FAISS).
- **Mindset:** Product-engineering shipper mindset. Experience building recommendation systems, NLP, or search at scale in product companies.
- **Experience Band:** 5–9 years of experience (ideal: 6-8 years). Must write production Python code.
- **Locations:** Preferred hybrid in Pune/Noida; welcome within India (Hyd/Mumbai/Delhi NCR). Relocation penalty for international candidates.
- **Notice Period:** Short notice periods preferred (sub-30 days ideal).

How We Evaluate Fit Beyond Simple Keyword Matching:

1. **Temporal Validation:** Matches candidate experience start dates against startup company founding histories (traps fake startup engineers).
2. **Skill Trust Multiplier:** Multiplies a skill weight by proficiency (expert=1.0, beginner=0.2) and the logarithm of the duration used (capping at 3 years), validating actual experience over keyword presence.
3. **Current Title Technical Relevancy:** Scans career history titles for technical keywords, penalizing keyword stuffers whose active title is non-technical (e.g. 'Marketing Manager').

4. Ranking Methodology

Candidates are evaluated dynamically out of **115 maximum points** based on the following formula components:

Component	Formula & Signal Logic	Max Pts
Experience (YoE)	6–8 years (Peak: +15 pts); 5–9 years (+12 pts); 4–5 or 9–11 years (+8 pts); 3-4 or 11-13 (+4 pts).	15
Skills Match	Skill Score = min(45, Σ [Weight * Proficiency Multiplier * Log10(duration + 1)/Log10(37) * 5.0]). - Weight 3.0: RAG, Vector DBs, Embeddings. - Weight 2.0: Python, PyTorch, LoRA/PEFT, eval metrics.	45
Location Alignment	Noida/Pune (+10 pts); Delhi NCR/Mumbai/Hyd (+5 pts); India + relocate (+5 pts); Outside India (-20 pts).	10
Notice Period	≤ 30 days (+10 pts); ≤ 60 days (+5 pts); 90 days (0 pts); > 90 days (-10 pts).	10
Active Engagement	Active ≤ 30d (+5 pts); Inactive > 6mo (-15 pts); Recruiter response rate ≥ 80% (+10 pts) or < 20% (-15 pts); Open to Work (+5 pts); Github ≥ 60 (+5 pts); Attendance ≥ 80% (+3 pts) or < 50% (-10 pts).	20
Career Relevance	Product company tenure ≥ 75% (+10 pts); Job Hopper tenure < 18mo (-8 pts); Current AI/ML Title match (+10 pts) or backend/software (+5 pts).	15

5. Explainability & Data Validation

Explainability through Custom Narratives:

- **Natural Reasonings:** Instead of duplicate or templated text, `rank.py` dynamically writes customized 1-2 sentence narratives summarizing the candidate's exact YoE, current employer, specific matching skills, location, and notice constraints.
- **Rank-Tone Matching:** Top-10 candidates get glowing justifications. Top-50 candidates get strong reviews with balanced notes. Bottom-100 candidates acknowledge specific drawbacks (e.g. high notice period).

Data Validation & Suspicious Profile Defense:

- **Zero Honeypots:** Automatically isolates all **84 honeypot candidates** (76 startup date violations and 8 skills traps) and sets their score to `-9999.0` immediately.
- **Anti-Hallucination:** Because the system reads variables straight from the parsed candidate record, it never generates fake skills, names, or employer names in the reasoning string.
- **IT Services-Only Filter:** Hard-filters candidates whose entire career was spent at consulting firms, guaranteeing they do not waste space in the top 100.

6. End-to-End Workflow

Step	Operation Description	Input / Output Data
1. Load Pool	Streams candidates.jsonl line-by-line using Python standard json. Parses dict structures.	In: raw jsonl lines. Out: dict list.
2. Sanitizer Filters	Evaluates startup timelines, empty expert skills, services-only histories, and non-tech titles.	Checks 84 honeypots and consulting traps.
3. Scoring Engine	Applies scoring heuristics for YoE, weighted skills match, location, notice period, and active engagement metrics.	Out: raw scores out of 115 pts.
4. Decimator & Round	Rounds scores to 1 decimal place to align output and sorting keys, avoiding artificial tie issues.	Out: rounded score (e.g. 120.0).
5. Sorter & Reasoning	Sorts by score (desc) and candidate_id (asc). Generates custom narrative summaries for the Top 100.	Ties broken lexicographically by ID.
6. CSV Generation	Writes columns (candidate_id, rank, score, reasoning) to the output CSV file.	Out: submission.csv (100 data rows).

7. System Architecture

The visual architecture of the automated candidate ingestion and ranking system is structured below:

Ingestion Layer	Sanitizer (Trap Filters)	Scoring Engine (Heuristics)	Ranking Layer
Stream candidates.jsonl line-by-line	Honeypot detector (startup timelines)	Experience Scorer (target 5-9 YoE)	Top-100 Slice
Parse Profile, Skills, and Signals dicts	Consulting-only filter (disqualify TCS/Wipro)	Log-Scaled Skills Scorer (RAG, Embeddings, Py)	Deterministic tie-breaker (candidate_id asc)
Zero memory overhead	Title validation (disqualify non-tech)	Location & notice period bonuses	Dynamic Narrative Generator
Stream size: 100k lines	**Status: 84 Traps Caught**	Behavioral & Activity signals	**Output: submission.csv**

8. Results & Performance

Rank	CID	Name	Current Title & Company	YoE	Location	Score
1	CAND_0061257	Advaith Pillai	Staff ML Engineer @ LinkedIn	8.0	Noida (Preferred)	**122.0**
2	CAND_0018499	Aarav Trivedi	Senior ML Engineer @ Zomato	7.2	Noida (Preferred)	**120.0**
3	CAND_0052328	Vikram Banerjee	Recommendation Systems Eng @ Amazon	6.5	Pune (Preferred)	**120.0**
4	CAND_0007009	Anika Pillai	Recommendation Systems Eng @ Wysa	7.9	Noida (Preferred)	**118.0**
5	CAND_0026942	Sneha Nair	Junior ML Engineer @ Verloop.io	6.0	Chandigarh (India)	**117.0**

Compute Performance Highlights:

- **Execution Time:** Completed in **3.4 seconds** for all 100,000 candidates on a standard CPU thread (well below the 5-minute constraint limit).
- **Memory Usage:** Consumed only **60 MB of RAM** at peak. Avoided bulk array loads into memory, allowing execution on tiny micro-nodes.
- **Ground Truth Compliance:** Honeypot rate is **0%** in the Top 100. Verification suite successfully passed.

9. Technologies Used

The system is engineered using highly lightweight, standard, and robust tools to guarantee sandboxed reproducibility:

Technology	Purpose	Selection Rationale
Python 3.11+	Core pipeline development, JSON stream parser, and mathematical heuristic engine.	Guarantees clean, readable code. Easy sandbox reproduction without compilation steps.
Standard Library	Parsing JSON files (`json`), reading CLI args (`argparse`), date matching (`datetime`), and writing CSV (`csv`).	Zero external dependencies, removing the risk of package mismatches or compile failures.
ReportLab 4.0	Programmatic PDF compilation (`platypus` flowables and custom canvas drawings).	Enables dynamic generation of presentation decks and reports straight from code.
Git & GitHub	Version control, repo management, and sharing project assets with organizers.	Maintains a clear iteration history and enables collaborative code reviews.

10. Submission Assets

All completed challenge materials are structured and uploaded per the hackathon requirements:

Asset File / Link	Description / Location
GitHub Code Repository	Contains full source code, scripts, requirements, and metadata: https://github.com/BharathKumarpallem/India_runs_data_and_ai_challenge
Hosted Sandbox Space	A working hosted Streamlit/Colab environment for small-sample reproduction: https://huggingface.co/spaces/bharathkumarpallem/redrob-ranker
Submission CSV	Located at `submission.csv` at the repo root. Contains exactly 100 validated rows.
Presentation PDF Slides	Located at `presentation.pdf`. Explains the Problem, Dataset, Method, and Results.
Honeypot Analysis Report	Located at `analysis_results.md` at the repo root. Details the 84 trapped profiles.

redrob | H2S

INDIA.RUNS

Build what next India runs on

THANK YOU